

PilotFish HTTP Post Transport – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

This is the general-purpose **HTTP POST client** at the end of a route. It takes the transaction body, POSTs it to a URL (as a raw body or as one form field), applies headers / auth / SSL / proxy rules, then puts the **HTTP response body** back into the transaction (and can fire a Response Listener).

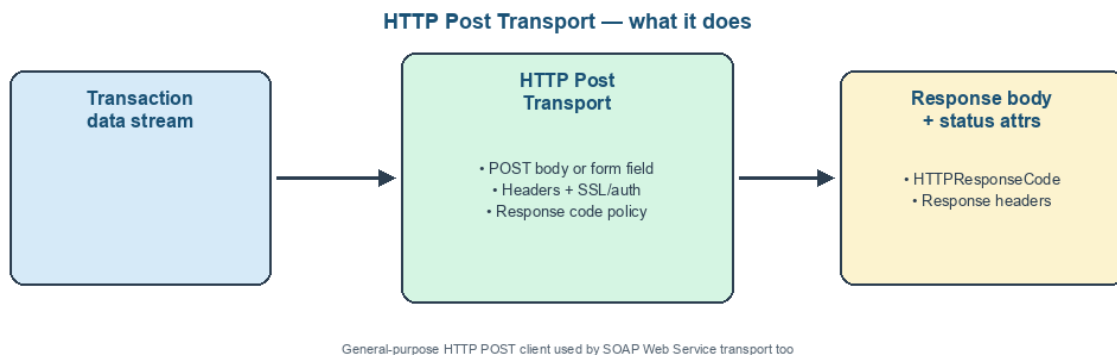


Figure 1: At a glance (plain English)

- **POSTs** payload bytes to **Target URL** (Apache HttpClient).
- **Controls** success/failure via Response Code Handling.
- **Captures** status and response headers as transaction attributes.
- **Optionally** synchronizes concurrent posts on a shared key.

Purpose

Deep dive into the **HTTP Post** transport in **eip.war.hs.26R1.11: sendData** caching/sync, Apache POST execution, redirect follow, response-code policy, and SSLAuthUtil options.

Provenance	Value
WAR	eip.war.hs.26R1.11
Module JAR	xcs-modules-http-1.0.3.jar
Decompiler	CFR 0.152
Implementation class	com.pilotfish.eip.modules.http.HttpPostTra
Decompiled path	war-pipeline/decompiled/eip-26R1.11/.../Ht
Transport type string	HTTP Post
Description	Sends data to a specified remote server/URL via an HTTP POST request.
Help ID	console.transport.httppost
Categories	HTTP, WEB

Also the base class for **SOAP Web Service** transport.

1. Conceptual model

`sendData:`

```
    cache inbound stream
    [optional sync on SynchronizationKey]
    executeTransport
```

`executeTransport:`

```
    read bytes -> HttpPost(URI)
    body = InputStreamEntity OR UrlEncodedFormEntity(FormElement)
    apply HTTPHeaders + StoredHeaders map
    SSL / proxy / basic auth / timeout
    execute
    evaluate status (ResultCodeHandling)
    [3xx] follow Location with another POST of original bytes
    copy response body to cache -> data stream
    optional Response Listener trigger
```

2. High-level behavior

2.1 sendData (overrides AbstractTransport)

1. If Synchronization Key blank: copy stream to default cache, `executeTransport`.

2. Else: obtain shared lock object for that key (refcount map), **synchronized** cache+execute, then **clearSynchronizationKey**.
3. On exception: error state; message includes Target URL.
4. On success: **setTransactionStateComplete**.

2.2 executeTransport

1. Read all inbound bytes; build URI with URL Param Encoding.
2. If Form Element empty: **InputStreamEntity**; honor Explicitly Disable Chunking.
3. Else: URL-encoded form with single name=Form Element, value=payload string (Form Element Encoding).
4. Add headers from HTTP Headers table + attribute map **com.pilotfish.eip.modules.http.StoredHeaders**.
5. Build Apache client with SSLAuthUtil context/hostname verifier.
6. Optional proxy via RequestConfig + NTCCredentials for proxy user/pass tags.
7. Optional Basic auth (+ Preemptive Authentication interceptor).
8. Timeout: if **Timeout > 0**, **setTimeout** uses **timeout * 1000** as ms for connect and socket (UI tooltip says “milliseconds” but code treats config as **seconds**).
9. Execute; set attributes:
 - **com.pilotfish.eip.modules.http.HTTPPostTransport.HTTPResponseCode**
 - **com.pilotfish.HTTPResponseCode**
10. If **resultIsError** -> IOException with built error message (Error Response Encoding).
11. If 3xx with Location: release, POST original bytes to redirect URL; set **com.pilotfish.HTTPPostRedirectRe** fail if still not 2xx.
12. Store response headers (flat vs Old Response Header Attributes mode).
13. Copy response bytes into response cache.
14. On soft failure path (Error on Connection Failure = false): write RouteTriggerMonitor error XML into response cache instead of throwing.
15. Finally close input; release connection.
16. Set data stream from response cache; trigger Response Listener if named.

resultIsError

Handling	Error when
Default	status < 200 or >= 400
Error Only On Listed Codes	code string appears in Response Codes list
Succeed Only On Listed Codes	code not in list
Pass Through	never (still may process redirects separately)

List matching uses **list.contains(String.valueOf(result))** – substring hazard for codes like 20 vs 200.

3. Configuration reference

3.1 Module-specific options

UI label	Tag	Type	Default	Tab	Required	Nuances
Target URL	TargetURL	String	""	Basic	Yes	Parsed / tokenized
User Name	UserName	String	""	Credential	No	With Password enables Basic
Password	Password	Password	""	Credential	No	
Auth Host	AUTH_HOST	String	""	Credential	No	Empty -> ANY_HOST
Auth Port	AUTH_PORT	String	""	Credential	No	Int; <=0 -> -1
Preemptive Authentication	PreemptiveAuth	Boolean	false	Credential	No	BasicScheme interceptor
Response Listener	ResponseListener	Listener	""	Response	No	Programmable
Response Code Handling	ResponseCodeHandling	Handler	Default	Response	No	See table above; includes Pass Through
Response Codes	ResponseCodes	String	""	Response	No	CSV; depends on listed-code modes
Timeout	Timeout	Integer	-1	Basic	No	-1/0 = no setTimeout; >0 treated as seconds
Error on Connection Failure	ErrorOnConnectionFailure	Boolean	false	Response	No	false -> error XML stream
Use Old Response Header Attributes	OldResponseHeaders	Boolean	false	Response	No	Element parsing vs flat header attrs
Synchronization Key	SynchronizationKey	String	""	Advanced	No	Serializes concurrent sendData
Form Element	FormElement	String	""	Advanced	No	Empty = raw body POST
HTTP Headers	HTTPHeaders	Table	Content- Type text/xml; charset=UTF- 8	Advanced	No	Name/value pairs
Explicitly Disable Chunking	DisableChunking	Boolean	false	Advanced	No	entity.setChunked(false)

UI label	Tag	Type	Default	Tab	Required	Nuances
Use Proxy	useProxy	Boolean	false	Proxy	No	Gates host/port
Proxy Host	proxyHost	String	127.0.0.1	Proxy	No	Read with empty Trans-actionData
Proxy Port	ProxyPort	Integer	8080	Proxy	No	Tag capital-P Port
Proxy Authentication Username	proxyUsername	String	""	Proxy	No	
Proxy Authentication Password	ProxyPassword	String	""	Proxy	No	Tag capital-P Password
URL Param Encoding	UrlParamEncoding	String	UTF-8	Encoding	Yes	URI builder
Form Element Encoding	FormElementEncoding	String	UTF-8	Encoding	Yes	Form field value charset
Error Response Encoding	ErrorResponseEncoding	String	UTF-8	Encoding	Yes	Error body serialization
Test Connection	button	Button	—	Basic	No	TestHttpConnection action

3.2 SSL (SSLAUTHUtil.getSSLConfigItems)

UI label	Tag	Default	Tab	Nuances
Trust self-signed certs	TrustSelfSignedCerts	false	Advanced	TrustStrategy
Validate SSL Hostnames	ValidateHostnames	true	Advanced	Custom verifier when false
Specify TLS Version	SpecifyTLSVersion	false	Advanced	Gates TLS Version
TLS Version	TLSVersion	TLSv1.1	Advanced	TLSv1.1 / TLSv1.2
KeyStore Path	KeyStoreFilePath		Keystore	
KeyStore Type	KeyStoreFileType	PKCS12	Keystore	PKCS12, JKS, JCEKS
KeyStore Password	KeyStoreFilePassword		Keystore	
Use KeyStore Attribute	UseCertificateAttribute	false	Keystore	
KeyStore Attribute	CertificateAttribute		Keystore	Shared tag name with Trust attr
TrustStore Source	TrustStoreSource	JVM	Truststore	JVM / OS / File / Attribute
TrustStore Type/Password/File/Attribute	TrustStoreFileType		Truststore	Depend on source

readConfiguration calls SSLAuthUtil.adjustForBackwardsCompatibility.

3.3 Inherited AbstractTransport

Overridden `sendData` still performs `checkRequiredData` via inner path; completion/error states managed in override. `supportsBatchSynchronization()` remains false (custom Synchronization Key instead).

4. Transaction attributes

4.1 Written

Attribute key	When
<code>com.pilotfish.eip.modules.http.HTTPPostTransport.HTTPResponseCode</code>	After response
<code>com.pilotfish.HTTPResponseCode</code>	Same status as string
<code>com.pilotfish.HTTPPostRedirectResponseCode</code>	After redirect POST
<code>com.pilotfish.http.response.<Header></code>	Default header mode
<code>com.pilotfish.http.response.<H>.<elem>...</code>	OldResponseHeaders mode
<code>com.pilotfish.http.ResponseHeaders</code>	HttpResponse object when old mode

4.2 Consumed

Attribute key	Usage
<code>com.pilotfish.eip.modules.http.StoredHeaders</code>	Extra request headers (Map)

5. Worked example – form-field POST with succeed-only 200

- Target URL: `https://api.example.com/ingest`
- Form Element: `payload`
- Response Code Handling: Succeed Only On Listed Codes / 200
- Content-Type header: `application/x-www-form-urlencoded`
- Timeout: 30 (seconds in practice)

Body "abc" becomes form field `payload=abc`. Non-200 throws. Response body replaces stream.

6. Known quirks and caveats

1. **Timeout unit mismatch** – tooltip “milliseconds”; code multiplies by 1000.
2. **Proxy config read with new TransactionData()** – tokens/attrs on live tx ignored for proxy host/port/user/pass.
3. **Redirect re-POST** may drop Form Element mode (always `InputStreamEntity` of original bytes).

4. **Response code list substring** matching.
5. **Pass Through** skips `resultIsError` but 3xx redirect branch still runs.
6. **Synchronization Key** nests only `executeTransport`; refcount cleaned after success path inside sync.

7. Operational recommendations

Goal	Guidance
Raw XML/JSON APIs	Leave Form Element empty; set Content-Type header
Strict SLA	Set Timeout in seconds ; Error on Connection Failure true
Debugging soft fails	Temporarily set Error on Connection Failure false to capture error XML
Parallel safety	Shared Synchronization Key when partner requires serial posts

8. Source index (this WAR)

Topic	Path under war-pipeline/decompiled/eip-26R1.11/
HTTP	<code>com/pilotfish/eip/modules/http/HttpPostTransport.java</code>
Post transport	
SSL util	<code>com/pilotfish/eip/modules/http/SSLAuthUtil.java</code>
HTTP utils	<code>com/pilotfish/eip/modules/http/HttpUtils.java</code>
Abstract transport	<code>com/pilotfish/eip/extend/AbstractTransport.java</code>
Module JAR	<code>war-pipeline/jars/eip-26R1.11/xcs-modules-http-1.0.3.jar</code>

9. Pipeline provenance

PilotFish WARs/eip.war.hs.26R1.11

-> xcs-modules-http-1.0.3.jar

-> CFR decompile

-> Documents/Transports/26R1.11/PilotFish-HTTP-Post-Transport-Reference-26R1.11.(md|pdf)

Deep-dive documentation from WAR decompile – HTTP Post Transport – 26R1.11 – August 2026.